



Text splitting strategies



Deepali Srivastava

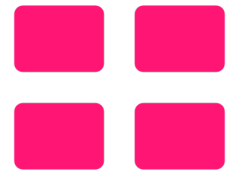
Author

[linkedin.com/in/deepali-srivastava/](https://www.linkedin.com/in/deepali-srivastava/)

Why text splitting is required



Limitations of LLM context windows



Loaded text cannot be directly sent to the LLM



LLMs have a limited context window



Large text must be divided into smaller chunks



The “lost-in-the-middle” problem

Models focus more on
the beginning and
end of long text

Important middle
information may be
overlooked

Smaller chunks
improve focus and
attention



Better embeddings with smaller chunks



Single idea per chunk

Focused text produces clearer vector representations



Improved embedding precision

Smaller, meaningful chunks generate higher-quality embeddings



Splitting improves retrieval

Retrieval selects
relevant text for a
query

Large documents
reduce retrieval
precision

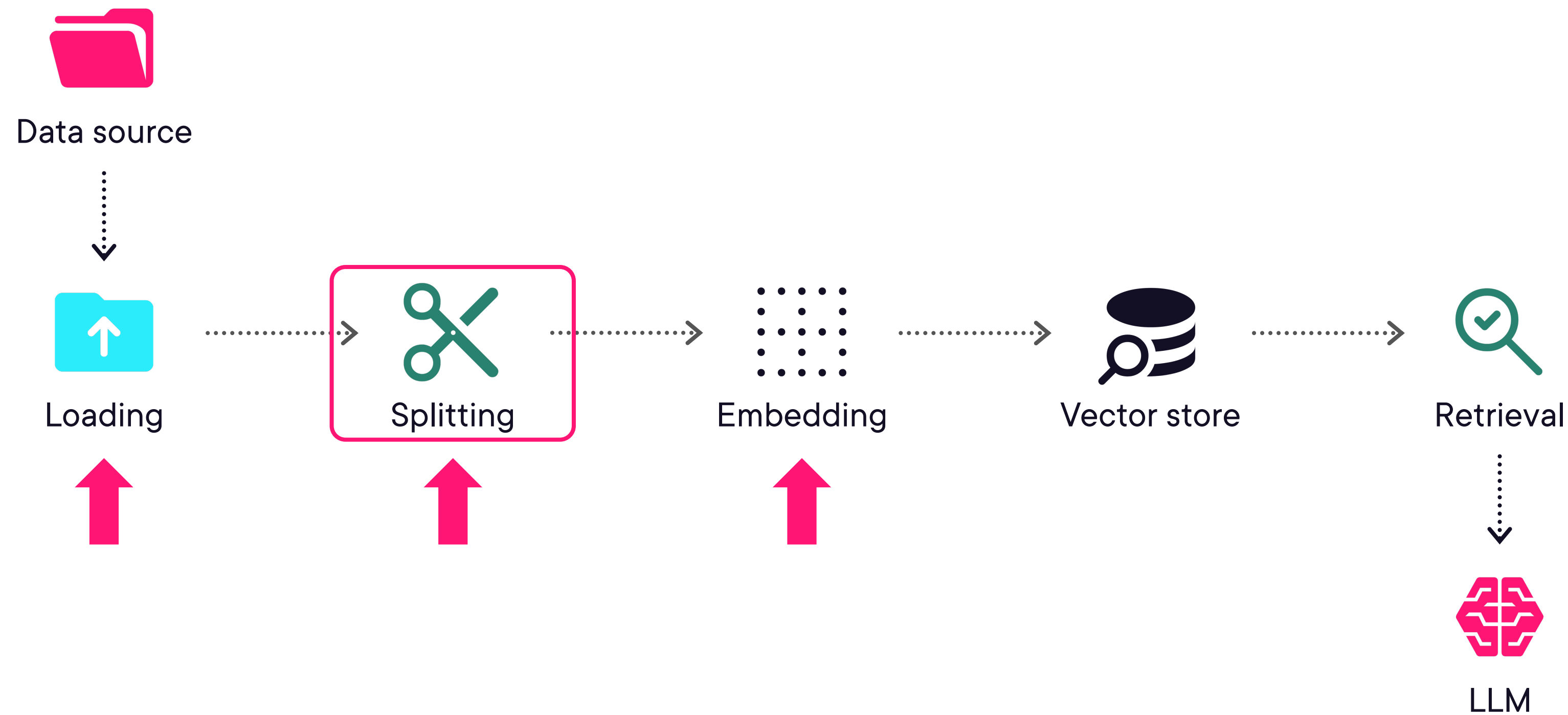
Smaller chunks enable
accurate retrieval



Introduction to text splitters in LangChain



Text splitters in the RAG pipeline



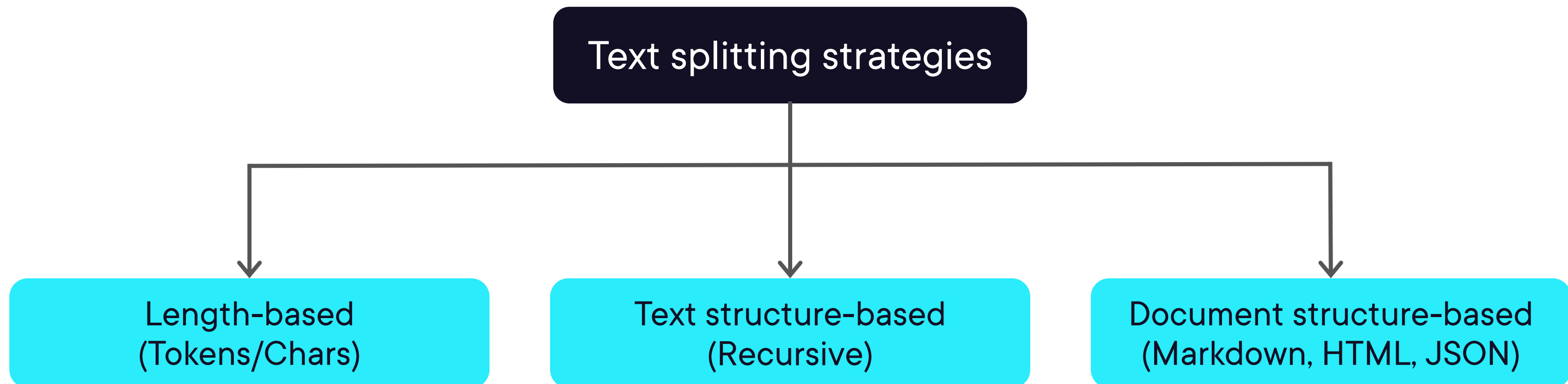
Separating loaders and splitters in LangChain

Loaders fetch documents, while
splitters transform them into
smaller chunks

This separation makes the
system modular, reusable, and
flexible across different data
sources



Common text splitting strategies in LangChain



CharacterTextSplitter for simple splitting



How CharacterTextSplitter works



Split by separator

Text is divided using a separator. By default, "\n\n" splits the text into paragraphs.

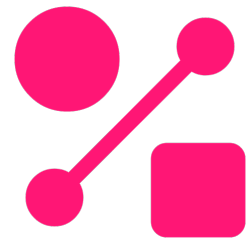


Merge pieces into chunks

Pieces are combined to form chunks until the chunk size limit is reached.

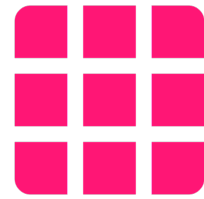


Parameters that control the behavior



separator

Defines where the text is split



chunk_size

Target size of each chunk in characters



chunk_overlap

Number of characters shared between adjacent chunks



CharacterTextSplitter in action

```
separator = "\n\n"  
chunk_size = 400
```

[[100 chars] "\n\n" [250 chars] "\n\n" [600 chars] "\n\n" [300 chars] "\n\n" [500 chars] "\n\n" [100 chars]

[100 chars] [250 chars] [600 chars] [300 chars] [500 chars] [100 chars]

Chunk 1: [100 chars] "\n\n" [250 chars] → [352 chars]



CharacterTextSplitter in action

```
separator = "\n\n"  
chunk_size = 400
```

[[100 chars] "\n\n" [250 chars] "\n\n" [600 chars] "\n\n" [300 chars] "\n\n" [500 chars] "\n\n" [100 chars]

[100 chars] [250 chars] [600 chars] [300 chars] [500 chars] [100 chars]

Chunk 1: [100 chars] "\n\n" [250 chars] → [352 chars]

Chunk 2: [600 chars] **oversize**

Chunk 3: [300 chars]

Chunk 4: [500 chars] **oversize**

Chunk 5: [100 chars]



Methods provided by the splitter

`split_text()` takes a string and returns text chunks

`create_documents()` converts raw text into Document chunks

`split_documents()` splits Document objects while preserving metadata



Demo

Splitting text using CharacterTextSplitter



Limitation of CharacterTextSplitter

Chunk size is not strictly guaranteed

Oversized chunks can create problems



RecursiveCharacterTextSplitter for preserving semantic boundaries



How RecursiveCharacterTextSplitter works

Uses multiple
separators to split text
`["\n\n", "\n", " ", ""]`

Recursively tries
smaller separators
until pieces fit within
the chunk size

Merges adjacent
pieces to create
chunks close to the
desired chunk size



Why the recursive strategy is useful



Strict size enforcement

Guarantees chunks stay within the chunk size limit



Context preservation

Preserves context by splitting with larger boundaries first



Demo

RecursiveCharacterTextSplitter in action



Choosing the right splitter

CharacterTextSplitter

Useful when the text already contains the specified separator frequently

RecursiveCharacterTextSplitter

Preferred for most cases because it preserves text structure and produces better chunks for RAG



Configuring chunk size and overlap



Deciding the best chunk size

Chunk size should be smaller than the embedding model's context window

Small chunks for precise fact retrieval, larger chunks for broader context

Start with a reasonable default and adjust based on retrieval performance



Understanding chunk overlap

Important information may lie at
chunk boundaries

Overlap repeats a small portion
of the previous chunk in the
next chunk



Choosing the overlap size



Larger overlap preserves more context but increases redundancy



Very small overlap is more efficient but may lose context



A common starting point is 10–20% of the chunk size



Structured documents usually need smaller overlap (5–10%)



Continuous or narrative text may need larger overlap (15–25%)



Demo

Chunking with overlap



Specialized splitters for structured content



Structure-aware text splitters



Many documents contain structure such as headings or HTML tags



Ignoring structure can break meaningful units



Structure-aware splitters preserve logical boundaries



Examples include splitters for Markdown, HTML, and source code



Demo

Splitting a markdown file



HTML structure-aware splitters

HTMLHeaderTextSplitter splits content using header tags (h1, h2, h3)

HTMLSectionSplitter splits HTML based on sections

HTMLSemanticPreservingSplitter preserves elements like tables and lists



Demo

Splitting an HTML file



Demo

Splitting a code file

